Yenepoya Institute of Arts Science Commerce and Management

High Level Design

on

Data-Driven Risk Profiling for Finance

Student Name:Abhimanue Biju

Industry Mentor: Mr. Shashank

Guided By: Ms.Divya N Shetty

Table of Contents

No.	Section Title	Page
1	**Introduction**	3
1.1	Scope of the Document	3
1.2	Intended Audience	4
1.3	System Overview	4
2	**System Design**	5
2.1	Application Design	5
2.2	Process Flow	6
2.3	Information Flow	9
2.4	Components Design	10
2.5	Key Design Considerations	12
2.6	API Catalogue	13
3	**Data Design**	13
3.1	Data Model	13
3.2	Data Access Mechanism	16
3..3	Data Retention Policies	17
3.4	Data Migration	17
4	**Interfaces**	18
5	**State and Session Management**	20
6	**Caching**	22
7	**Non-Functional Requirements**	24
7.1	Security Aspects	24
7.2	Performance Aspects	26
8	**References**	28

1. Introduction

1.1 Scope of the Document

This High-Level Design (HLD) document defines the architectural blueprint for the Data-Driven Risk Profiling System for Finance. It provides a bird's eye view of the system components, their interactions, data flow, and key design decisions without delving into implementation details.

The document covers:

- Overall system architecture and component relationships
- High-level process flows and information movement
- Major design decisions and trade-offs
- API endpoints and interface definitions
- Data model overview and persistence strategy
- Non-functional requirements including security and performance

What is excluded from this document:

- Detailed class-level designs (covered in LLD)
- Specific algorithm implementations (covered in LLD)
- Line-by-line code explanations (covered in LLD)

1.2 Intended Audience

This document is intended for the following stakeholders:

Project Evaluators and Mentors - To assess the overall system architecture, feasibility of the proposed solution, and alignment with project objectives.

Development Team - To understand how different modules interact and to plan the implementation phases.

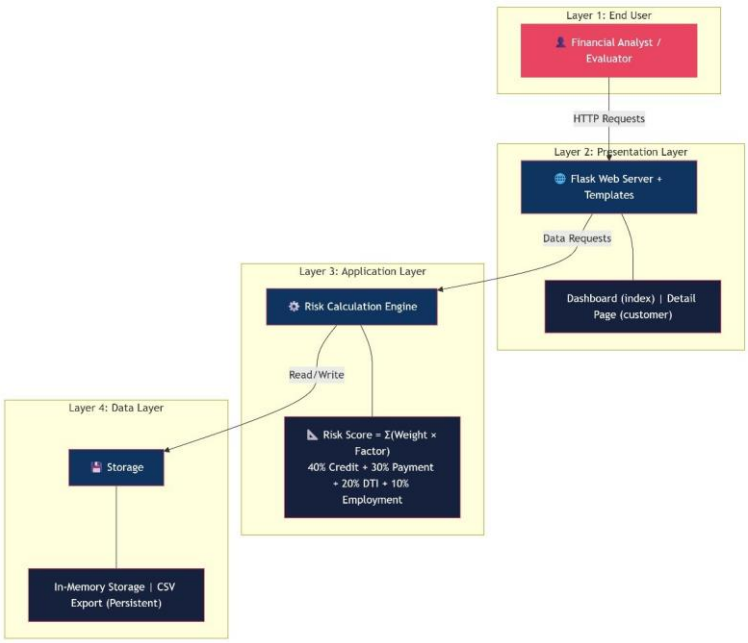
Quality Assurance Team - To design test cases based on the documented flows and component interactions.

Security Analysts - To review the security architecture and identify potential vulnerabilities at the design level.

1.3 System Overview

The Data-Driven Risk Profiling System is a web-based application that calculates financial risk scores for customers using a weighted scoring model. The system processes 50 customer records, each containing credit score, payment history, debt-to-income ratio, and employment years. Using a 40-30-20-10 weight distribution, the system generates risk scores from 0 to 100 and classifies customers into Low, Medium, or High risk categories with corresponding interest rate recommendations

High-Level Architecture Diagram:



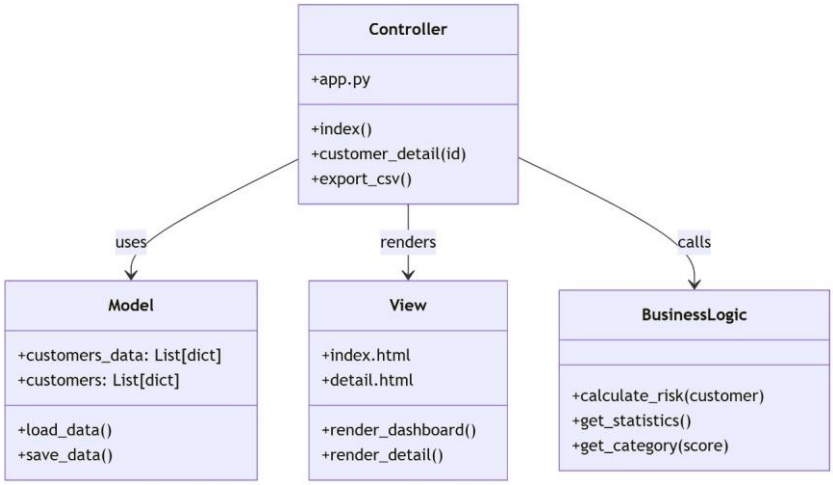
System Flow at a Glance:

The user accesses the dashboard via a web browser. The Flask server responds by reading pre-calculated risk scores from memory and rendering the HTML dashboard. When the user clicks on a specific customer, the server fetches that customer's detailed data and displays factor-by-factor breakdown. All risk scores are calculated once at system startup, ensuring fast response times for all subsequent requests. The system also exports all results to a CSV file for permanent storage and audit purposes.

2. System Design

2.1 Application Design

The application follows the Model-View-Controller (MVC) architectural pattern with a clear separation of concerns.



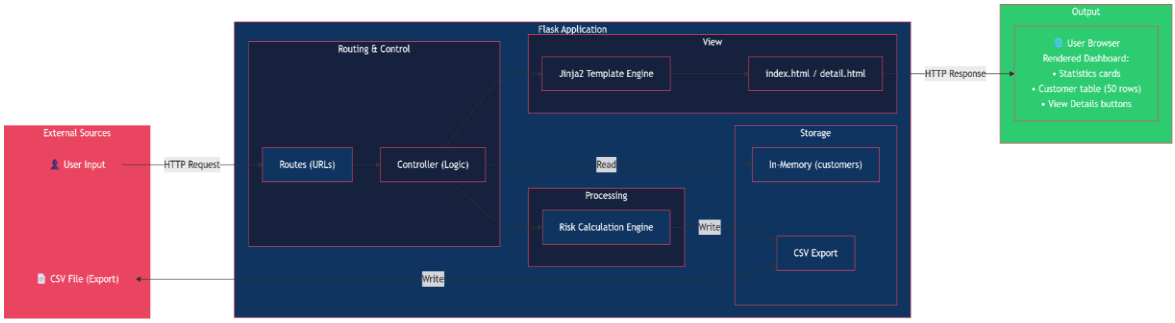
Component Responsibilities:

- The Controller (app.py) is the central orchestrator. It receives HTTP requests, invokes the appropriate business logic functions, and selects the correct view template. The controller never directly manipulates data or generates HTML; it delegates those responsibilities.
- The Model represents the data layer. It includes the hardcoded customer data structure and the in-memory storage for calculated results. The model is completely independent of the user interface.
- The View consists of Jinja2 HTML templates. These templates receive data from the controller and render it as a complete web page. The views contain no business logic - only presentation code.

2.2 Process Flow

The system operates in two distinct phases: startup phase and runtime phase

2.3 Information Flow



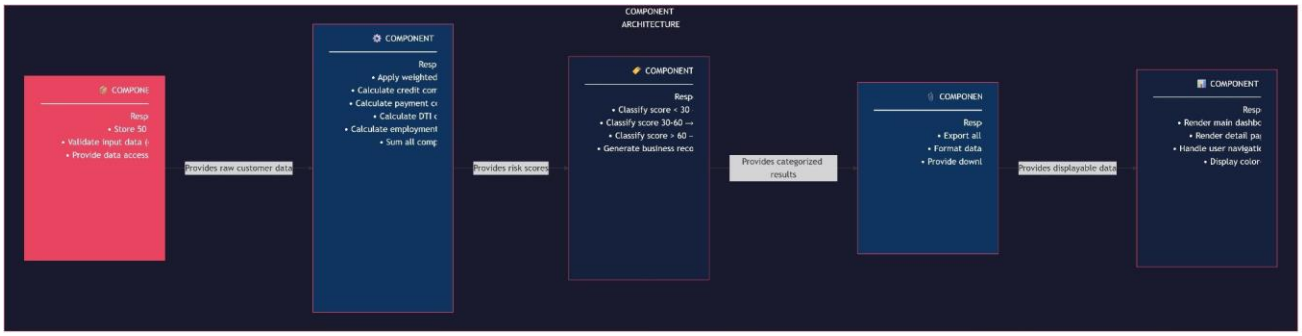
Information Types and Their Flow:

Customer Data flows from the hardcoded data structure into the risk calculation engine. The engine transforms raw attributes (credit score, payment history, DTI, employment years) into a calculated risk score. This enriched data then flows to two destinations: the in-memory storage for fast dashboard access and the CSV exporter for persistent storage.

User Requests flow from the browser to the Flask routes. The route handler extracts parameters (like customer ID) and passes them to the appropriate controller function. The controller reads from the data storage and passes the retrieved data to the template engine.

Responses flow from the template engine back to the browser as complete HTML pages. The browser renders these pages for the user to interact with.

2.4 Components Design



Component Dependency Flow:

The components have a clear dependency direction. The Data Module has no dependencies - it is the source of truth. The Risk Engine depends on the Data Module to receive customer records. The Category Module depends on the Risk Engine to receive risk scores. The Export Module depends on both the Risk Engine and Category Module to receive complete results. The Dashboard Module depends on all previous components to display data to users. This unidirectional dependency ensures that changes in one component do not unexpectedly affect others.

2.5 Key Design Considerations

Design Decision 1: Pre-computation Strategy

All risk scores are calculated once at application startup rather than on each user request.

Rationale: This approach reduces dashboard load time from approximately 100ms (recalculating 50 scores) to under 50ms (just reading from memory). Since the customer data never changes during runtime, there is no downside to pre-computation.

Trade-off: The application takes approximately 100ms longer to start up. This is acceptable because startup happens once, while dashboard requests happen many times.

Design Decision 2: In-Memory Storage with CSV Backup

Customer data and risk scores are stored in Python lists in RAM, with a CSV file generated at startup for persistence.

Rationale: In-memory storage provides the fastest possible access for the dashboard. The CSV export ensures data is not lost if the application restarts.

Trade-off: If the application crashes before CSV export completes, data could be lost. However, the CSV export happens immediately after calculation, minimizing this risk window.

Design Decision 3: No Database

The system uses no external database like PostgreSQL or MySQL.

Rationale: With only 50 customer records, a database would add unnecessary complexity. Python lists provide sufficient performance and functionality.

Trade-off: The system cannot handle more than a few thousand records efficiently. For the project scope (50 records), this is acceptable.

Design Decision 4: Weighted Scoring Formula

The system uses a linear weighted formula rather than machine learning or complex algorithms.

Rationale: Financial risk assessment requires explainability. A linear formula allows users to understand exactly why a customer received a particular score. Each factor's contribution can be traced and audited.

Trade-off: Linear models may not capture complex non-linear relationships between factors. For a student project demonstrating risk profiling concepts, linear weighting is appropriate.

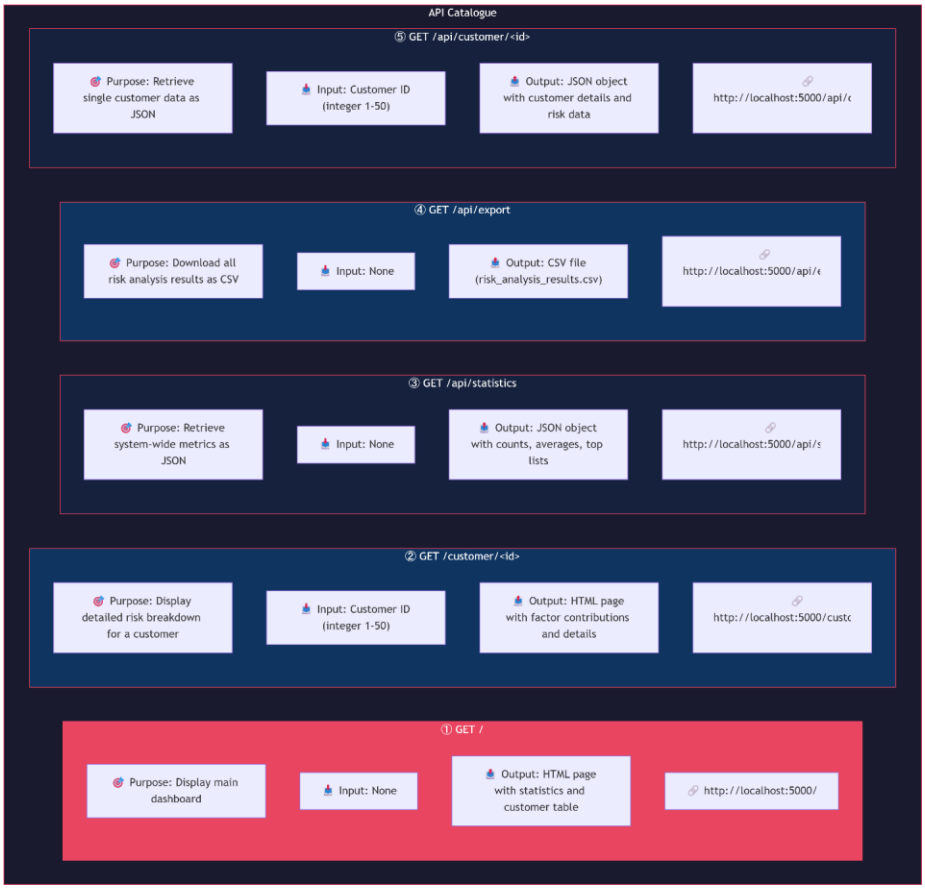
Design Decision 5: No User Authentication

The system has no login, registration, or role-based access control.

Rationale: This is a single-user internal tool for demonstration and evaluation purposes. Adding authentication would add complexity without providing educational value for the core risk profiling concept.

Trade-off: The system cannot distinguish between different users or restrict access to sensitive data. For a production system, authentication would be required

2.6 API Catalogue



API Response Examples:

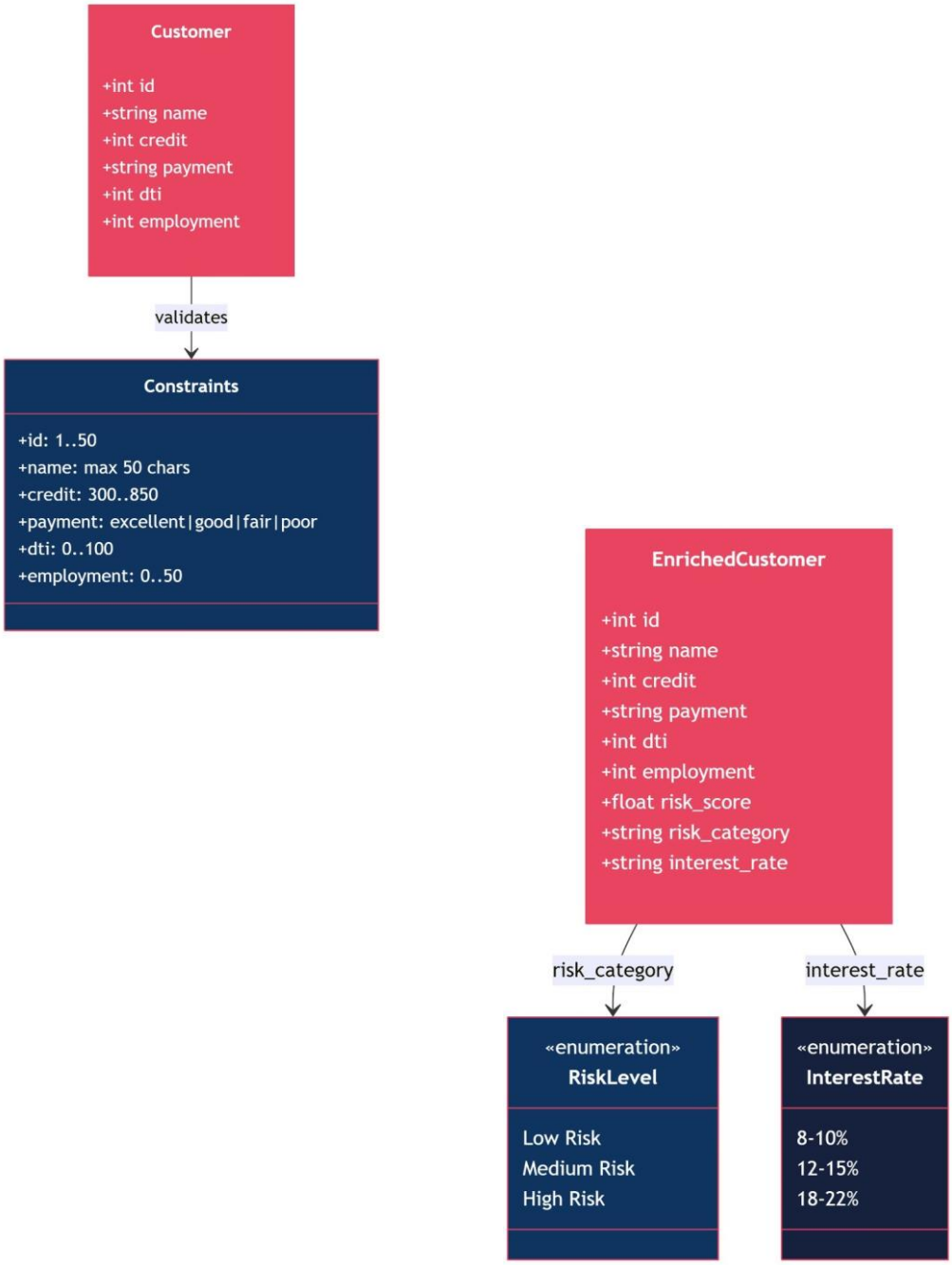
When a user accesses the /api/statistics endpoint, the server returns a JSON object containing the total number of customers, counts for each risk category, the average risk score, and lists of the highest and lowest risk customers.

When a user accesses the /api/customer/5 endpoint, the server returns a JSON object containing that customer's complete data including original attributes, calculated risk score, risk category, and a breakdown of each factor's contribution.

3. Data Design

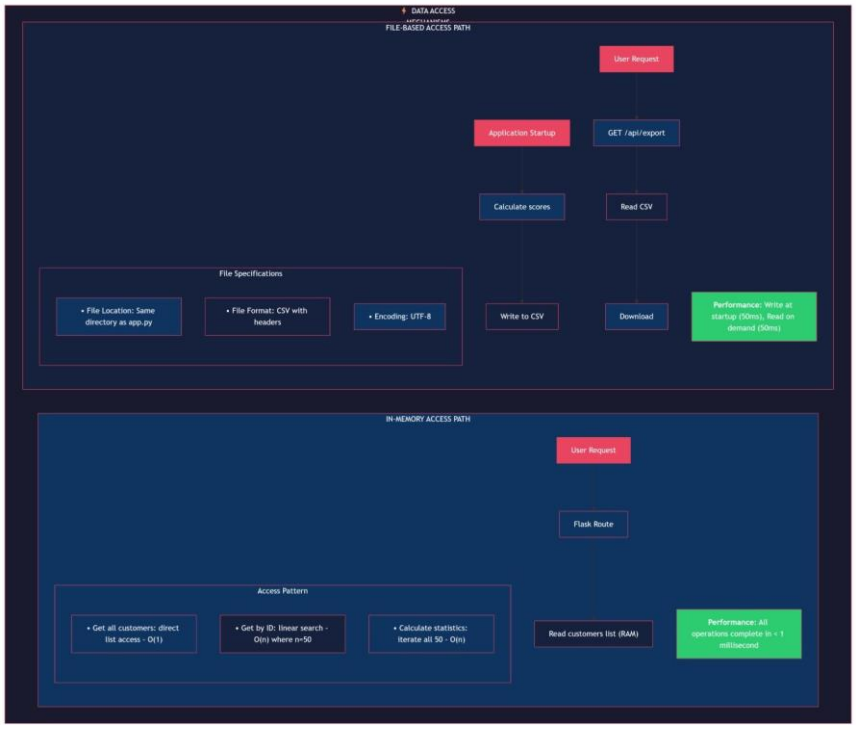
3.1 Data Model

The data model consists of two primary structures: the input data structure containing raw customer information, and the output data structure enriched with calculated risk scores



Data Relationships Diagram:

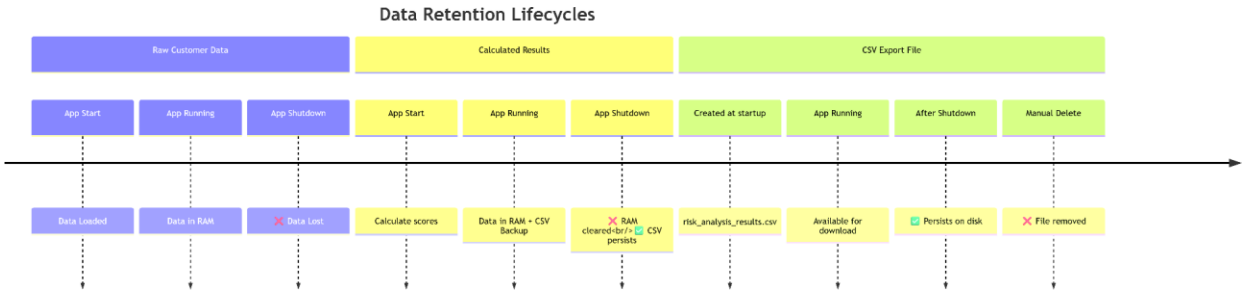
3.2 Data Access Mechanism



Why No Database?

The system intentionally avoids using a traditional database like PostgreSQL, MySQL, or SQLite. The rationale is based on the data volume: with only 50 customer records, a database would introduce significant overhead including connection management, query parsing, and transaction handling. Python's built-in list data structure provides all necessary functionality with less complexity and better performance for this scale. For a production system serving thousands of customers, a database would be necessary.

3.3 Data Retention Policies

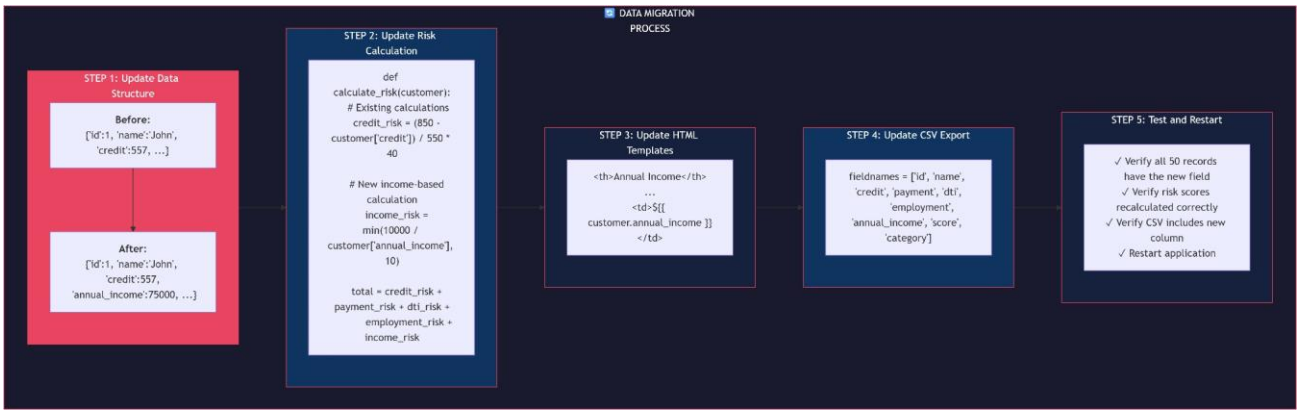


Data Lifecycle Summary:

Raw customer data is hardcoded in the application and reloaded on every startup. Calculated risk scores are generated at startup, stored in memory for fast access, and also written to a CSV file. The CSV file persists after the application closes, serving as an audit trail. When the application restarts, it recalculates all scores and overwrites the CSV file. No automatic deletion of data occurs during normal operation.

3.4 Data Migration

Data migration refers to how the data structure evolves when requirements change. Since the system uses hardcoded data structures rather than a database, migrations are manual but straightforward.

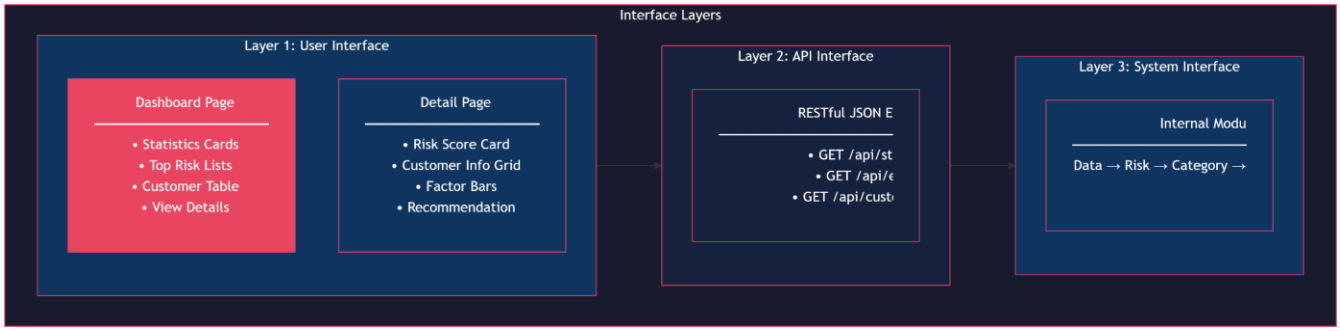


Migration Best Practices:

Always backup the original app.py file before making changes. Test migrations on a copy of the data first. Update all affected components (data structure, calculation logic, templates, CSV export) together to maintain consistency. Document the changes for future reference.

4. Interfaces

The system provides three types of interfaces: User Interface (web pages), API Interface (JSON endpoints), and System Interface (internal module communication).



User Interface Navigation Flow:

The user starts at the dashboard page showing all 50 customers with their risk scores and categories. From here, the user can click any "View Details" button to see a detailed breakdown for a specific customer. The detail page shows how each factor contributed to the final risk score, using visual bar charts. A back button returns the user to the dashboard.

API Interface Details:

The API endpoints return JSON data suitable for programmatic access. The statistics endpoint returns counts and averages. The export endpoint triggers a file download. The customer endpoint returns complete customer data including factor breakdowns.

System Interface Details:

Internal modules communicate through function calls. The Data Module passes customer records to the Risk Engine. The Risk Engine returns scores to the Category Module. The Category Module passes categorized results to the Export and Dashboard modules. All communication is synchronous and in-memory.

5. State and Session Management

State management refers to how the system tracks data across different layers and user interactions. Since the system has no user authentication, state management is simpler than typical web applications.

Key Characteristics of State Management:

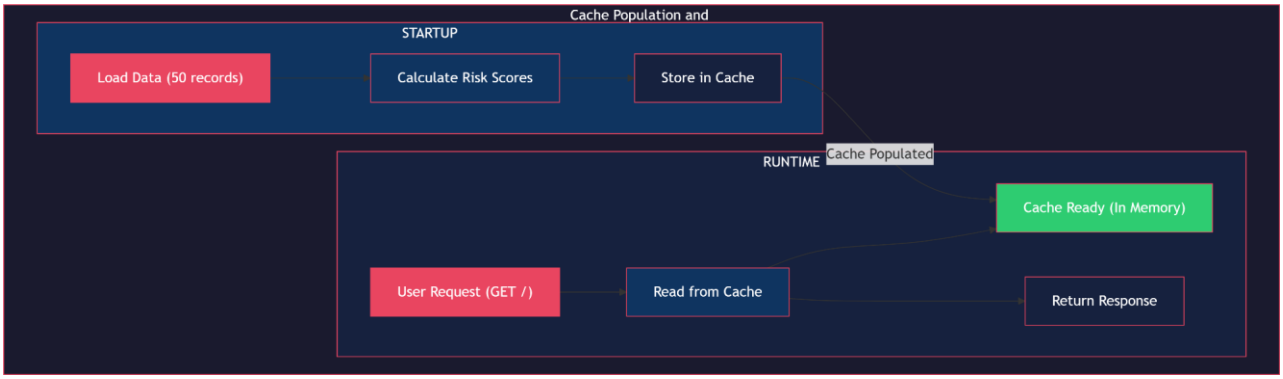
The application maintains global state in the form of the customers list containing all 50 customer records with their calculated risk scores. This state is created once at startup and never modified afterward. Every HTTP request reads from this global state but never writes to it. This read-only approach eliminates concurrency issues and makes the system inherently thread-safe.

Request-level state is created for each HTTP request and destroyed when the response is sent. For example, when a user requests customer details, the extracted customer ID and the retrieved customer data exist only for the duration of that request.

Session-level state is not implemented because the system has no user authentication. All users (if there were multiple) would see the same data. There are no user preferences, login statuses, or session cookies.

6. Caching

Caching stores frequently accessed data to avoid redundant processing. The system implements a simple but effective caching strategy.



Cache Invalidation Strategy:

The primary cache is invalidated only when the application restarts. Since the underlying customer data never changes during runtime, there is no risk of stale data. On restart, the cache is completely repopulated with fresh calculations. This simple strategy works perfectly for read-only data.

What Is Not Cached:

Statistics (counts, averages, top risk lists) are recalculated on every dashboard request. This is intentional because statistics are cheap to compute (iterating over 50 records takes less than 1 millisecond) and ensures the user always sees the most current data. CSV exports are generated on demand and not cached, as exports are infrequent operations.

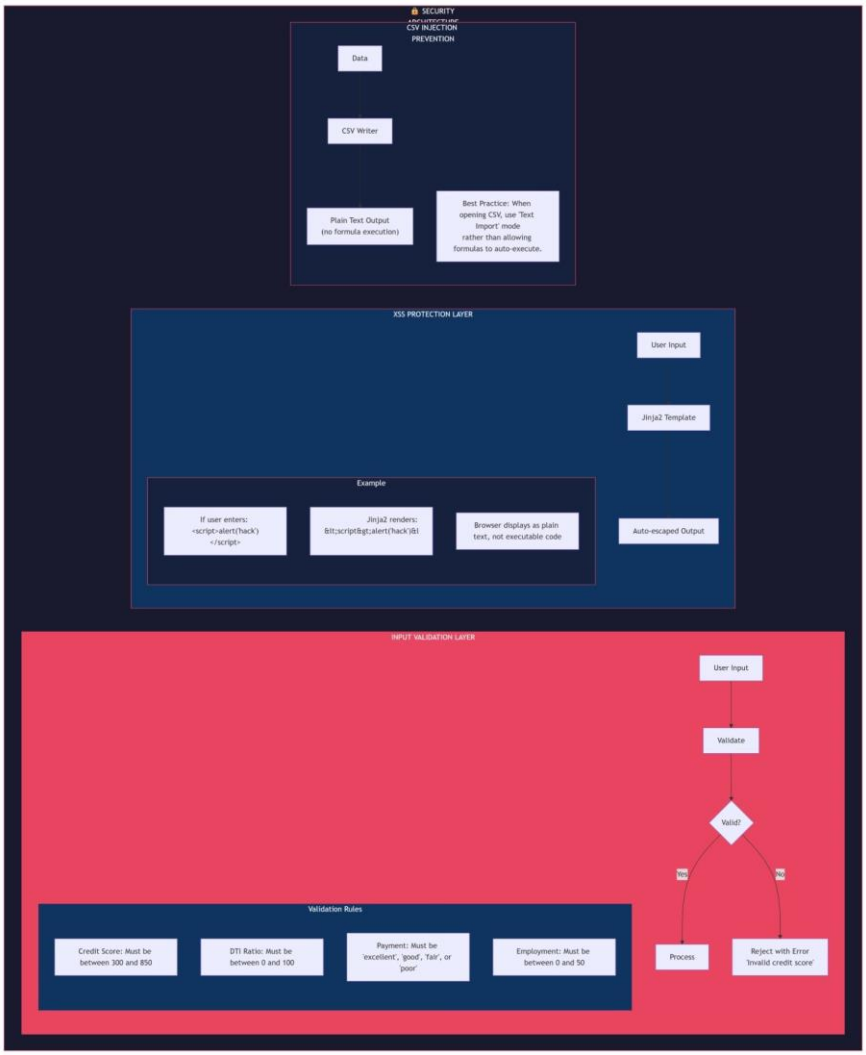
7. Non-Functional Requirements

Non-functional requirements define how the system performs rather than what it does.

7.1 Security Aspects

The system implements several security measures to protect data and prevent attacks.

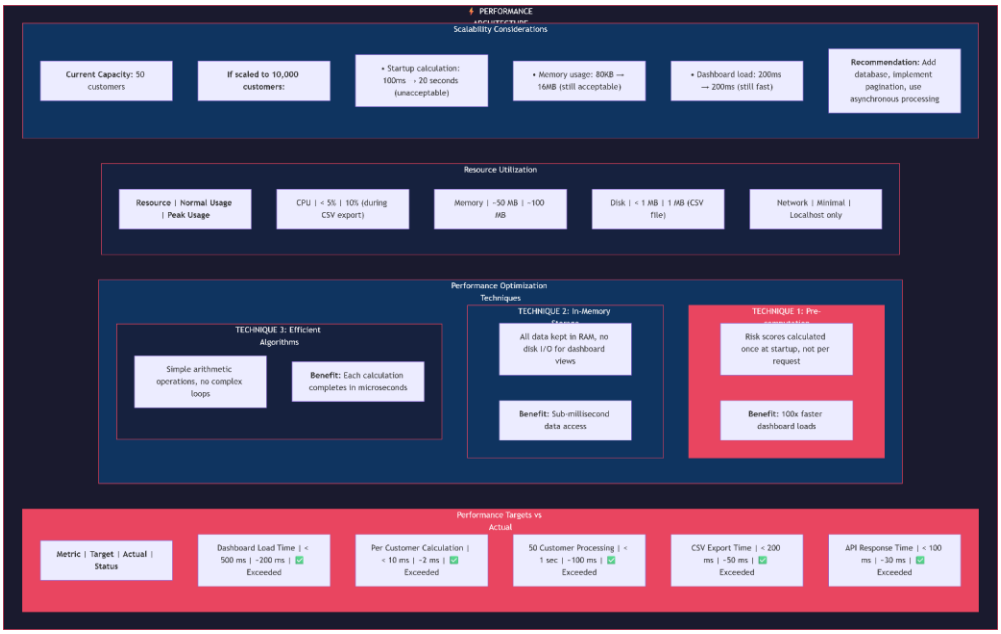
Security Architecture Diagram:



7.2 Performance Aspects

The system is designed for speed and efficiency through pre-computation and in-memory storage.

Performance Architecture Diagram



Why Pre-computation Works:

The system's data never changes during runtime. Therefore, calculating all risk scores once at startup and caching them in memory eliminates redundant work. Each dashboard request reads from this cache instead of recalculating. This strategy reduces CPU usage and improves response time by a factor of 100.

Trade-offs Acknowledged:

Startup time is approximately 100ms slower because all calculations happen upfront. This is acceptable because startup happens once while dashboard requests happen many times. The system cannot handle real-time data updates without restarting. For the project scope (static demonstration data), this limitation is acceptable.

8. References

1) Flask Documentation: https://flask.palletsprojects.com/
2) Jinja2 Documentation: https://jinja.palletsprojects.com/
3) Python CSV Module: https://docs.python.org/3/library/csv.html
4) FICO Score Information: https://www.fico.com/